

Autoencoders

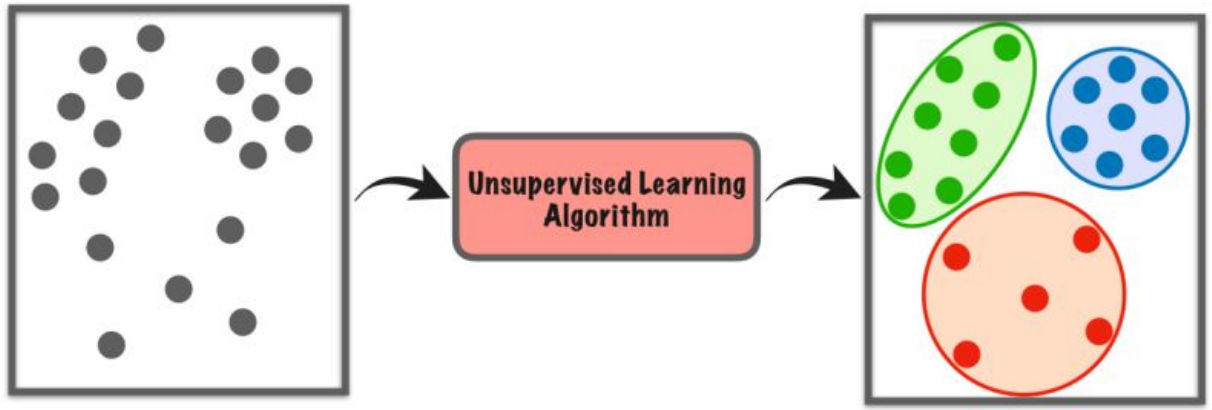
Prof. Sérgio Canuto - sergio.canuto@ifg.edu.br

Tópicos

- - Contextualização
- - Autoencoder
- - Autoencoder Convolutional
- - Autoencoder LSTM
- - Autoencoders Subcompletos x Sobrecompletos
- - Denoising Autoencoders
- - Aplicações

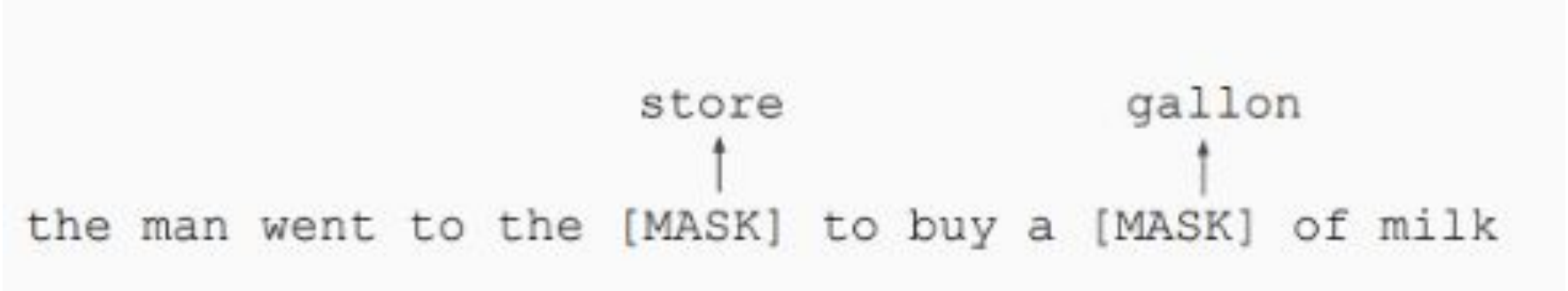
Contextualização: Métodos de Aprendizado não Supervisionado

- **Não há rotulação explícita:**
  - Em problemas de agrupamento, visamos encontrar grupos coesos e distantes uns dos outros.
  - De forma geral, métodos não supervisionados visam identificar sub-estruturas comuns nos dados sem supervisão/rotulação explícita.



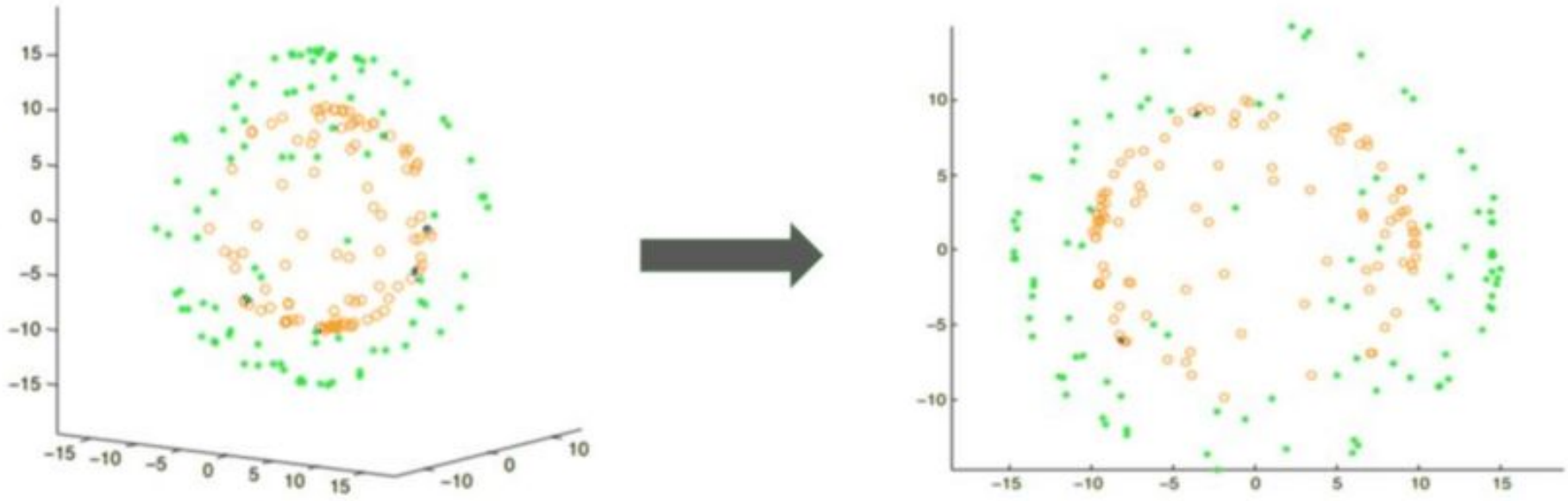
Contextualização: Métodos de Aprendizado não Supervisionado

- **Não há rotulação explícita:**
  - Estratégias mais recentes de deep learning são voltadas ao aprendizado de sub-estruturas/padrões textuais em grandes volumes de texto.
  - Idéia:
    - Se tenho uma boa representação de padrões textuais, então consigo "adivinhar" qual termo deveria estar na sentença, ou qual deveria ser a próxima sentença.
    - Tendo uma boa representação dos padrões de linguagem, é possível aplicar-la a outras tarefas não supervisionadas.



Contextualização:
Métodos de Aprendizado não Supervisionado

- **Não há rotulação explícita:**
  - PCA (Principal Component Analysis).
    - Abordagem estatística clássica para compressão de dados e visualização (Karl Pearson, 1901).
    - Limitação: componentes lineares.



Contextualização: Métodos de Aprendizado não Supervisionado

- **Não há rotulação explícita:**
  - Se conseguimos aprender como representar a essência de uma imagem de forma compactada, também conseguimos reconstruir tal imagem.

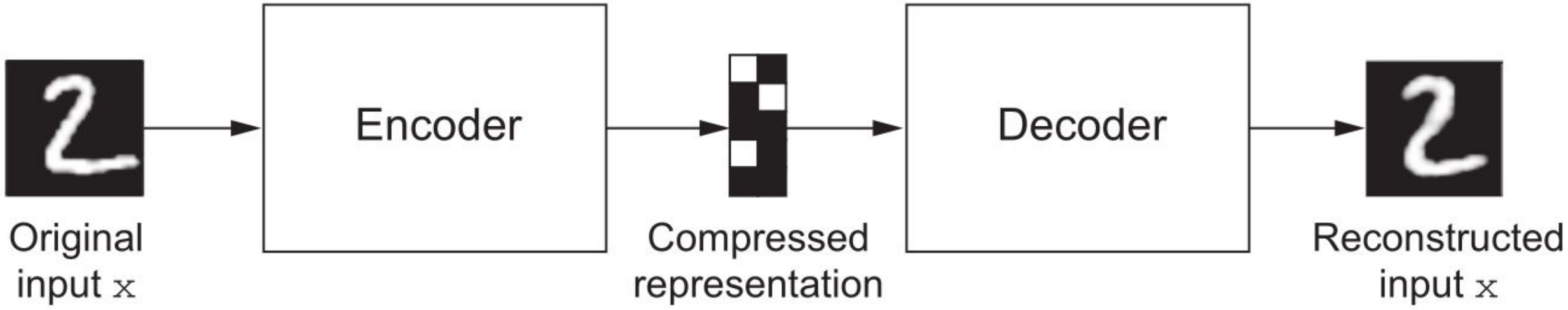


Figure 8.12 An autoencoder: mapping an input x to a compressed representation and then decoding it back as x'

Redes Autocodificadoras (*autoencoders*)

- Podem aprender a estrutura do conjunto de dados de forma não supervisionada.

- Aprendizado de representações mais concisas.
- Extrai o que é mais relevante para representar uma entrada:
  - Remoção/identificação de ruído.
- Detecção de outliers
  - https://datascience.oneoffcoder.com/autoencoder-anomaly.html
- Atributos identificados são úteis para uso posterior em aprendizagem supervisionada.

Transformações em Autoencoders

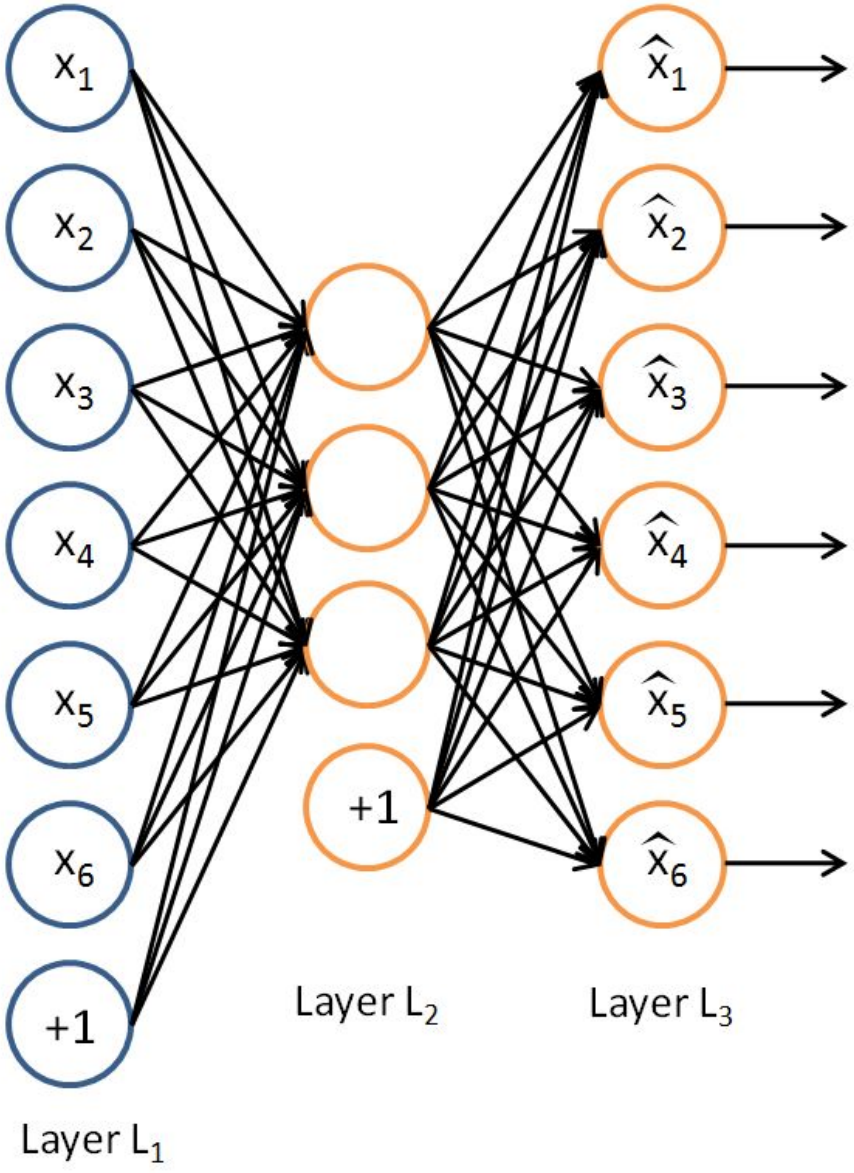
- Transformações são aplicadas na entrada de acordo com dois tipos de funções.
  - Função de extração de características (*encoder*) mapeia o conjunto de treinamento para uma **representação latente**.

$$h : \mathbb{R}^D \rightarrow \mathbb{R}^K$$

  - Função de reconstrução (*decoder*) mapeia a representação produzida por *h* de volta para o espaço original.

$$r : \mathbb{R}^K \rightarrow \mathbb{R}^D$$

Autoencoder Tradicional



Onde esperamos que $x_i \approx \hat{x}_i$

- Diferente de PCA, temos não-linearidade.
- Um Autoencoder sem funções de ativação (ativação linear) usualmente tem capacidade semelhante à do PCA.

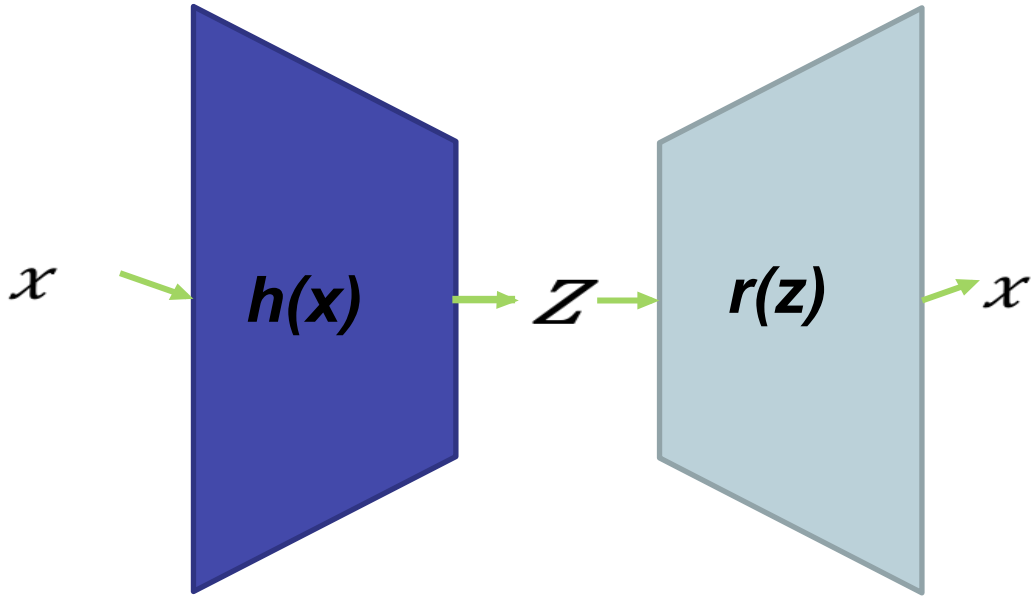
http://ufldl.stanford.edu/tutorial/unsupervised/Autoencoders/

Autoencoder Tradicional

- Seja *z* uma **representação latente** e *s* uma função de ativação não linear (como a sigmoid).

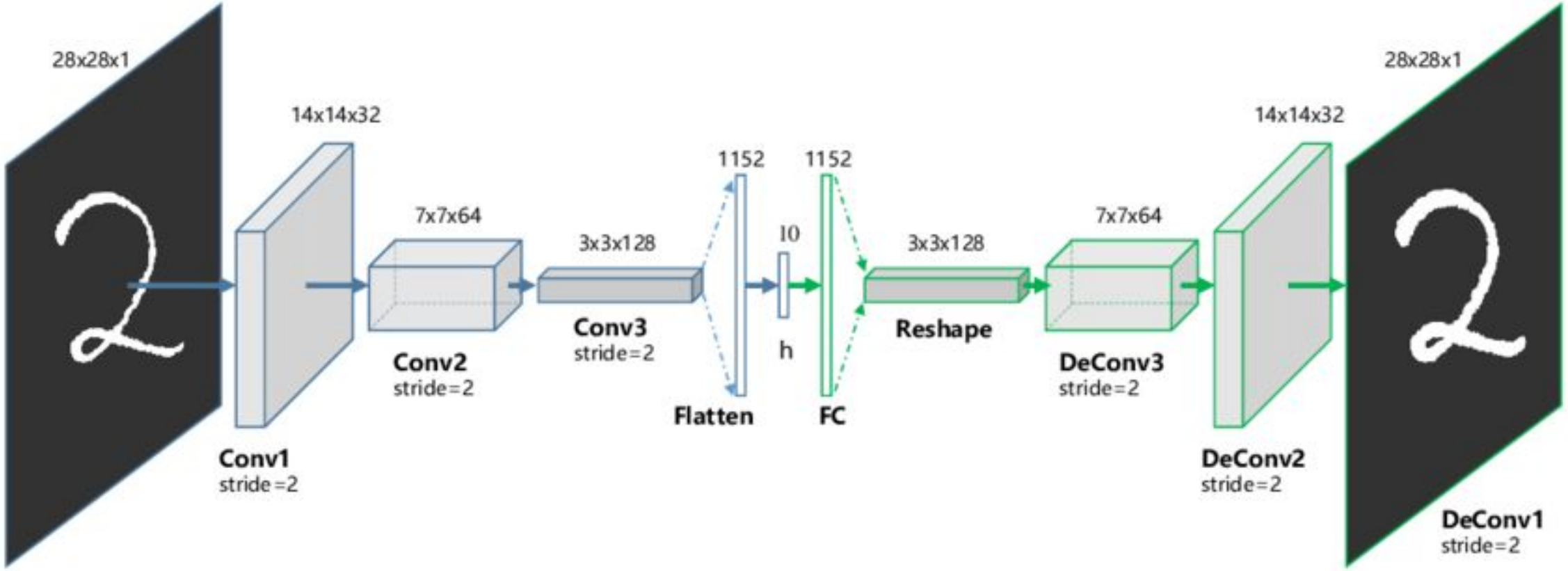
- Para um dado x de entrada devemos aprender as funções h (encoder) e r (decoder), onde:
  - *h(x) = s(wx+b) = z*
  - *r(z) = s(w'z+b') = x*

- *De forma que podemos definir:*
  - *a(x) = r(h(x)) = x'*
    - *onde a(x) é uma aproximação da função identidade.*

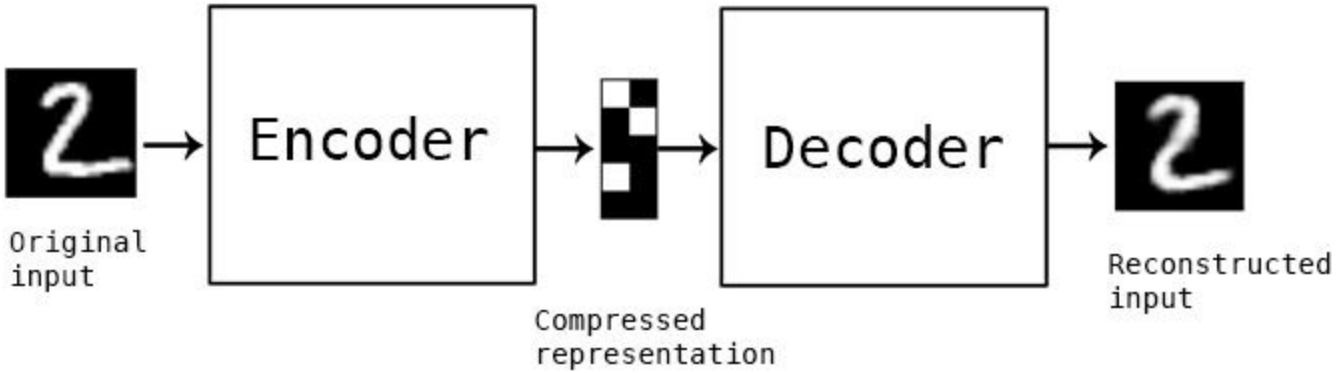


Autoencoder + Convolução

- Entradas normalizadas
- Todas as camadas com a ativação RELU, exceto a última (sigmoid)



Modelo Exemplo




```
import keras
from keras import layers
encoding_dim = 32
input_img = keras.Input(shape=(784,))
encoded = layers.Dense(encoding_dim, activation='relu')(input_img)
decoded = layers.Dense(784, activation='sigmoid')(encoded)
autoencoder = keras.Model(input_img, decoded)

...
autoencoder.fit(x_train, x_train,  epochs=50,  batch_size=256,  shuffle=True)
```

Notebook completo:

https://colab.research.google.com/drive/12_TKWZl_NtjA1du5QjC8t8pkKAScSOOR?usp=sharing

Modelo Exemplo (Múltiplas camadas):

```
input_img = keras.Input(shape=(784,))
encoded = layers.Dense(128, activation='relu')(input_img)
encoded = layers.Dense(64, activation='relu')(encoded)
encoded = layers.Dense(32, activation='relu')(encoded)

decoded = layers.Dense(64, activation='relu')(encoded)
decoded = layers.Dense(128, activation='relu')(decoded)
decoded = layers.Dense(784, activation='sigmoid')(decoded)

autoencoder = keras.Model(input_img, decoded)

autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
autoencoder.fit(x_train, x_train,epochs=100, batch_size=256)
```

Modelo Exemplo:

x_test: 

autoencoder.predict(x_test): 

- **Obtendo as representações compactadas:**

```
encoder = keras.Model(input_img, encoded)
encoded_imgs = encoder.predict(x_test)
```

Modelo Exemplo (Autoencoder + Convolução):

```
input_img = keras.Input(shape=(28, 28, 1))
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = layers.MaxPooling2D((2, 2), padding='same')(x)
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
encoded = layers.MaxPooling2D((2, 2), padding='same')(x)

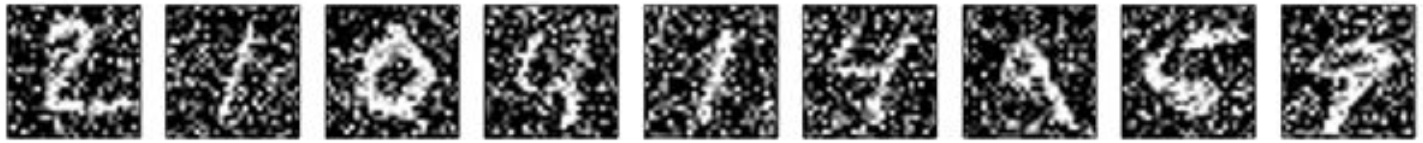
# 32 maps de ativação, 7x7

x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(encoded)
x = layers.UpSampling2D((2, 2))(x)
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = layers.UpSampling2D((2, 2))(x)
decoded = layers.Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)

autoencoder = keras.Model(input_img, decoded)
```

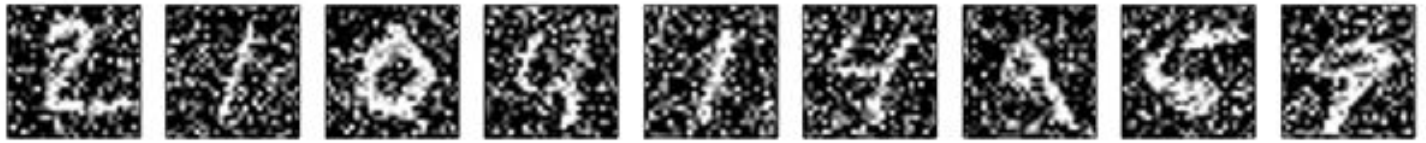
Modelo Exemplo (Autoencoder+Convolução):

E se passarmos uma entrada ruidosa?

x_test: 

Modelo Exemplo (Autoencoder+Convolução):

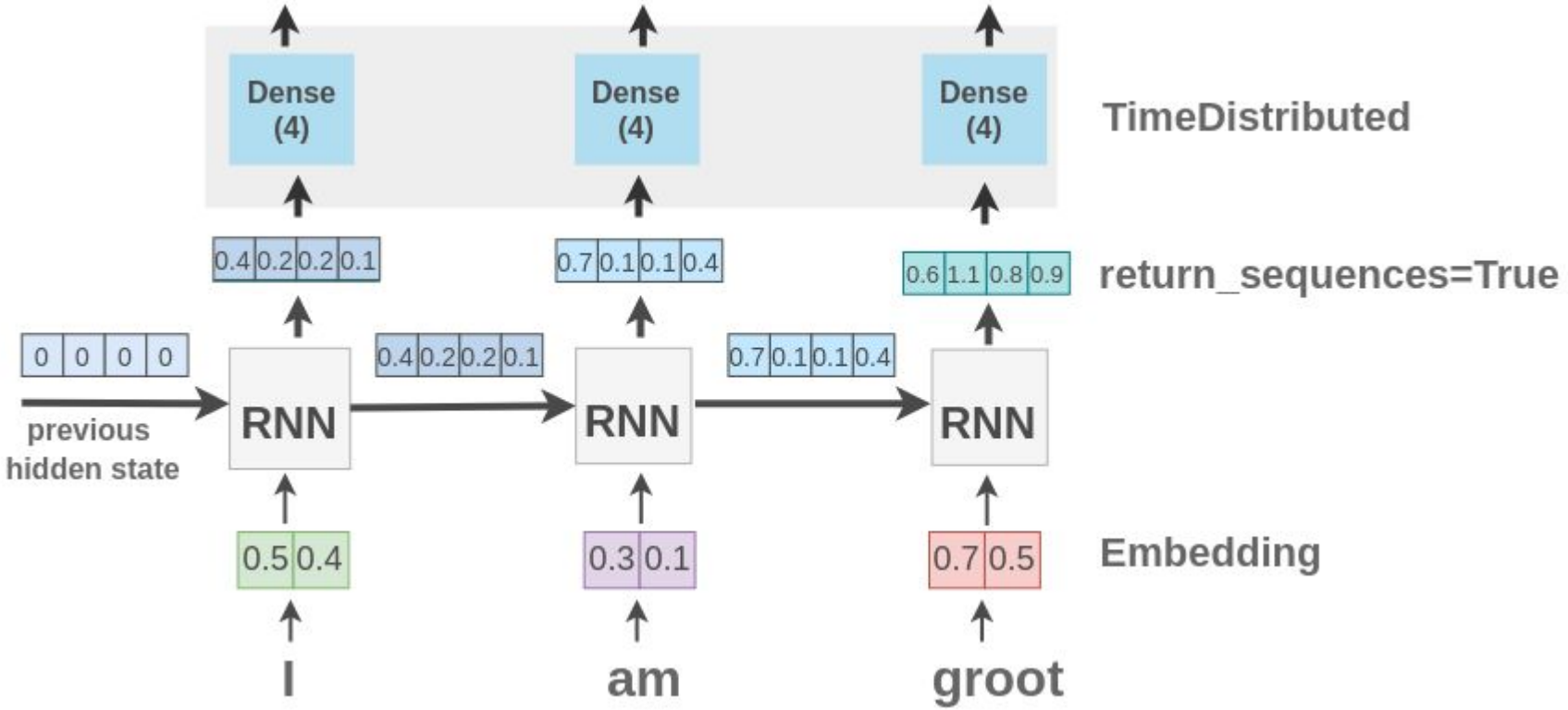
E se passarmos uma entrada ruidosa?

x_test: 

autoencoder.predict(x_test): 

O autoencoder é uma ferramenta útil para mitigar ruído na base de dados!

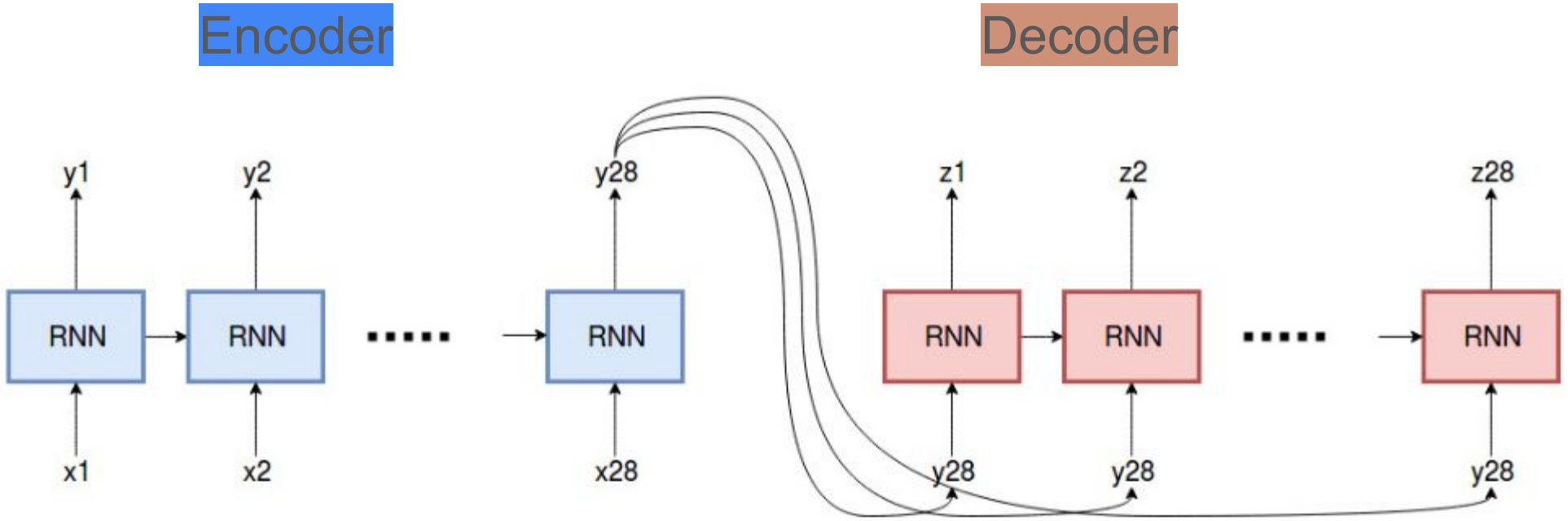
Modelo Exemplo (Autoencoder+LSTM): (lembrando da aula passada...)




```
model = Sequential()
model.add(SimpleRNN(4, input_shape=(3, 2), return_sequences=True))
model.add(TimeDistributed(Dense(4, activation='softmax')))
```

Modelo Exemplo (Autoencoder+LSTM):

- Nessa implementação, após processar toda a sequência de 28 elementos, temos o estado oculto da rede (y28).
- Esse mesmo estado oculto é passado 28x como entrada.
  - O que muda é somente o estado oculto a cada iteração da rede.
- Esperamos que z_1 reconstrua x_1, z_2 reconstrua x_2…



Modelo Exemplo (Autoencoder+LSTM):

```
timesteps = …
input_dim = …
latent_dim = ...

inputs = keras.Input(shape=(timesteps, input_dim))
encoded = layers.LSTM(latent_dim)(inputs)

decoded = layers.RepeatVector(timesteps)(encoded)
decoded = layers.LSTM(input_dim, return_sequences=True)(decoded)
decoded = TimeDistributed(Dense(input_dim)))(decoded)

sequence_autoencoder = keras.Model(inputs, decoded)
encoder = keras.Model(inputs, encoded)
```

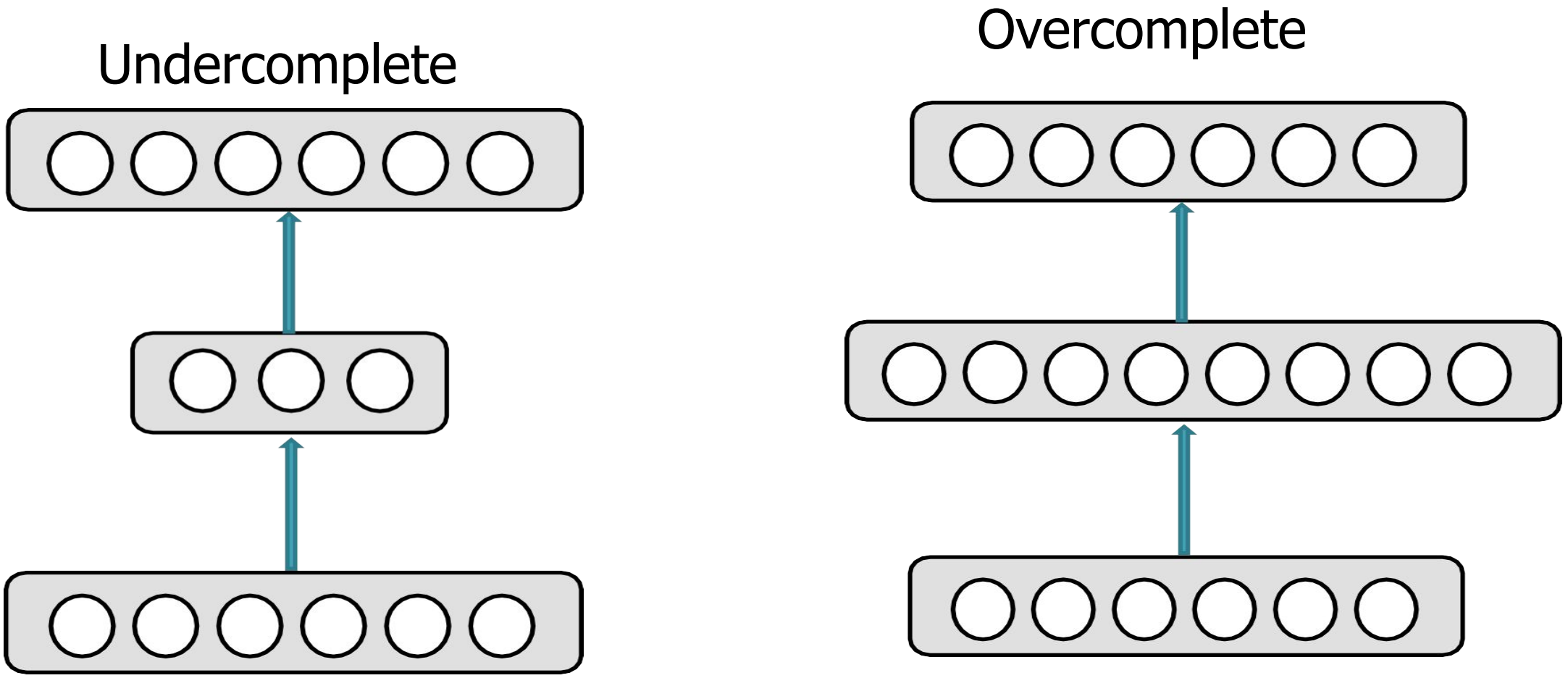
fonte: https://blog.keras.io/building-autoencoders-in-keras.html
https://towardsdatascience.com/step-by-step-understanding-lstm-autoencoder-layers-ffab055b6352

Autoencoders Subcompletos x Sobrecompletos (Undercomplete vs Overcomplete)

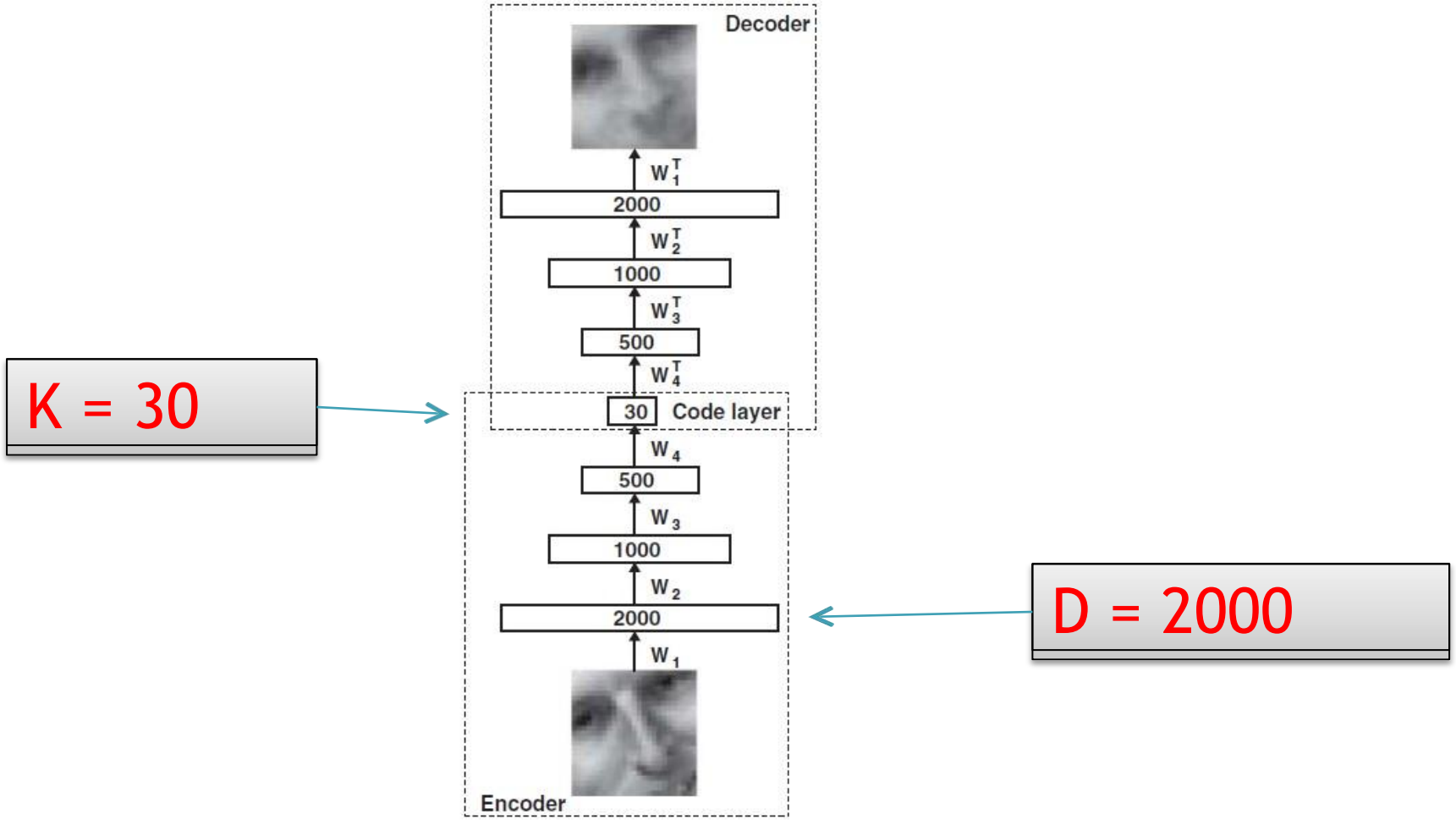
- Representação latente em um autoencoder tem dimensão K:
  - K < D: autoencoder subcompleto.
  - K > D: autoencoder sobrecompleto.

- A escolha de K determina
  1. Quantidade de unidades da camada intermediária (saída do encoder).
  2. Quantidade de informação que o autoencoder pode aprender da distribuição de entrada.

Autoencoders Subcompletos x Sobrecompletos (Undercomplete vs Overcomplete)



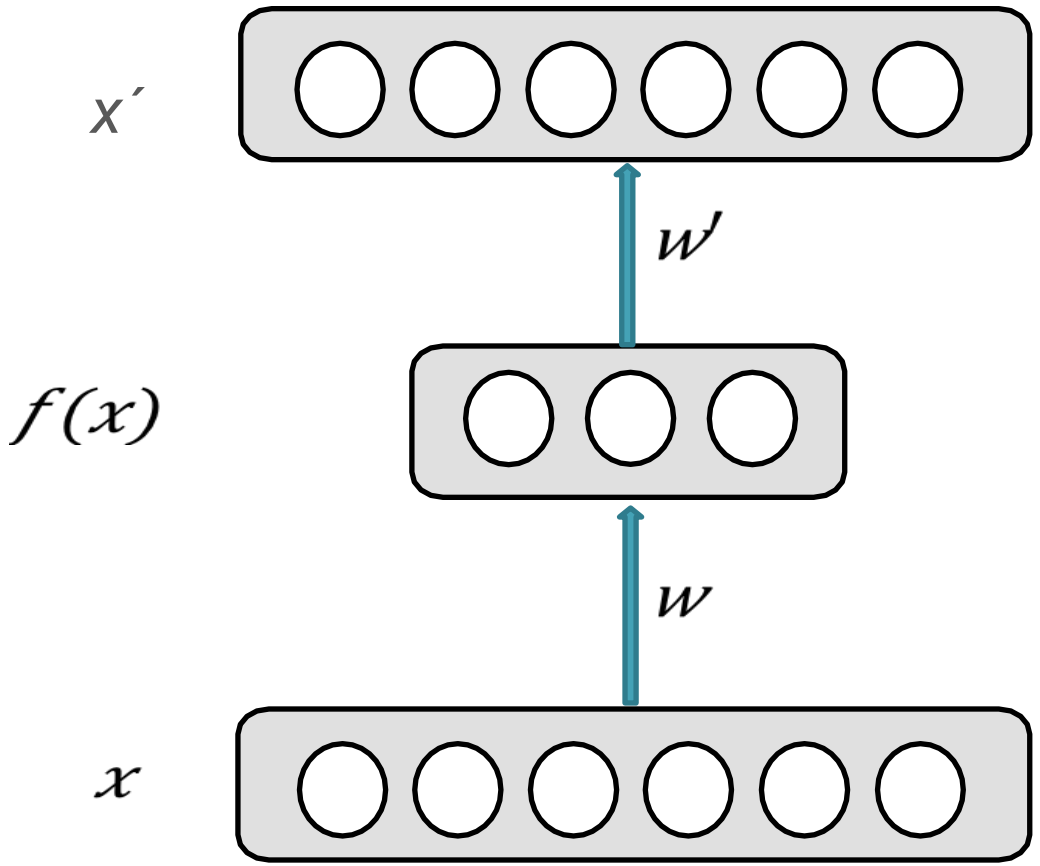
Autoencoder subcompleto:



Reducing the Dimensionality of Data with Neural Networks (Science, 2006)

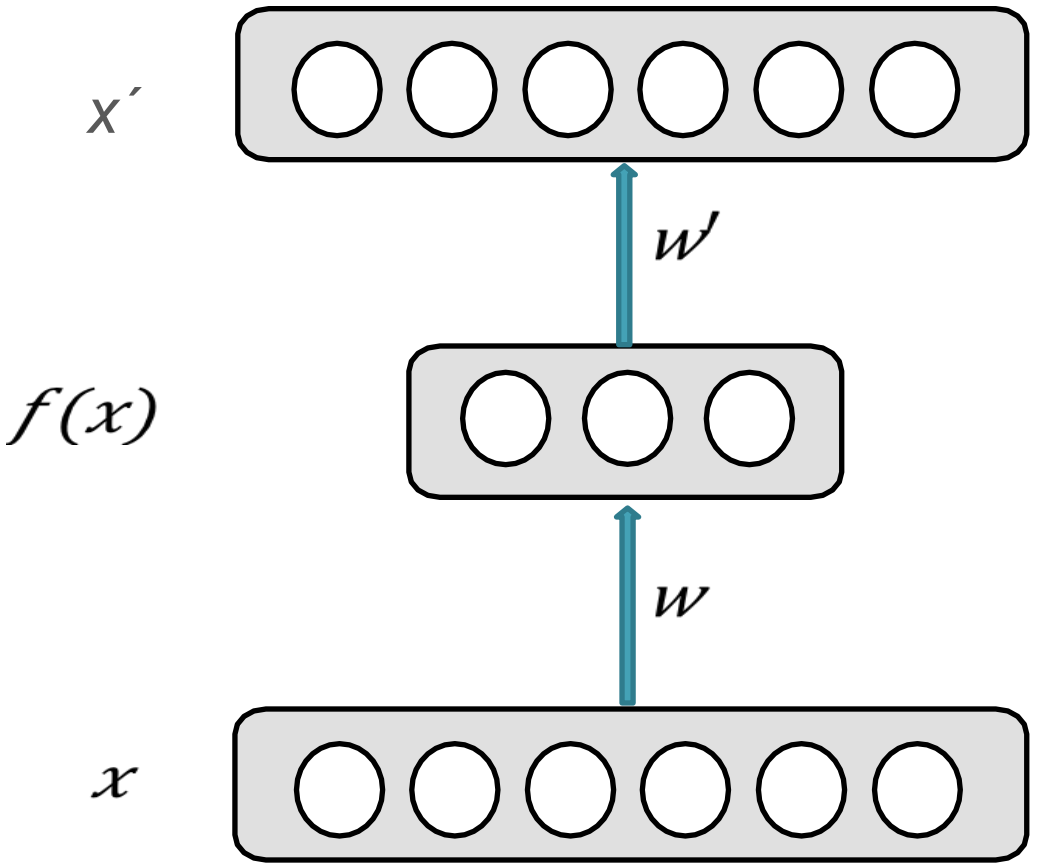
Autoencoder Subcompleto

- Camada escondida é **Subcompleta** se é menor que a de entrada
  - ❑ Comprime a entrada
  - ❑ Comprime bem apenas para conjunto a distribuição de dados de treinamento
- Nós escondidos formam
  - ❑ Boas características para distribuição de treinamento
  - ❑ Pode não generalizar bem (overfitting)



Autoencoder subcompleto: faz sentido copiar a entrada na saída?

- ~~Não estamos~~ interessados na cópia...
- Caso de interesse:
  - ❑ Não copiar exatamente, mas ter dificuldade em copiar!
  - ❑ Autoencoder se vê forçado a selecionar aspectos que podem representar propriedades úteis nos dados.
  - ❑ Proposto originalmente na tese do LeCunn: Y. LeCun. Connexionist learning models. PhD thesis, Universite Pierre et Marie Curie (Paris), 1987



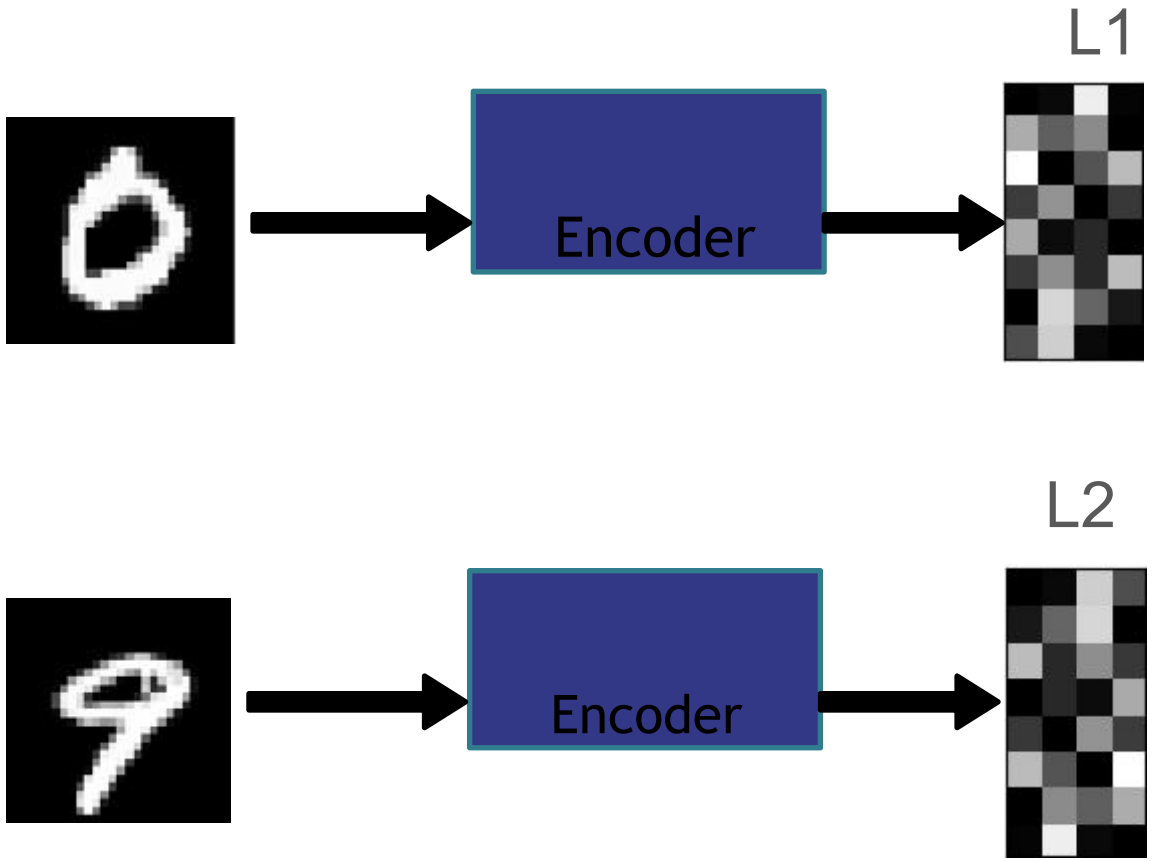
Autoencoder subcompleto: faz sentido copiar a entrada na saída?

- Modelos generativos:
  - ❑ Dado um conjunto de treino, gerar novas amostras da mesma distribuição
  - ❑ Variational Autoencoders (cap. 8, Deep Learning with Python de François Chollet)

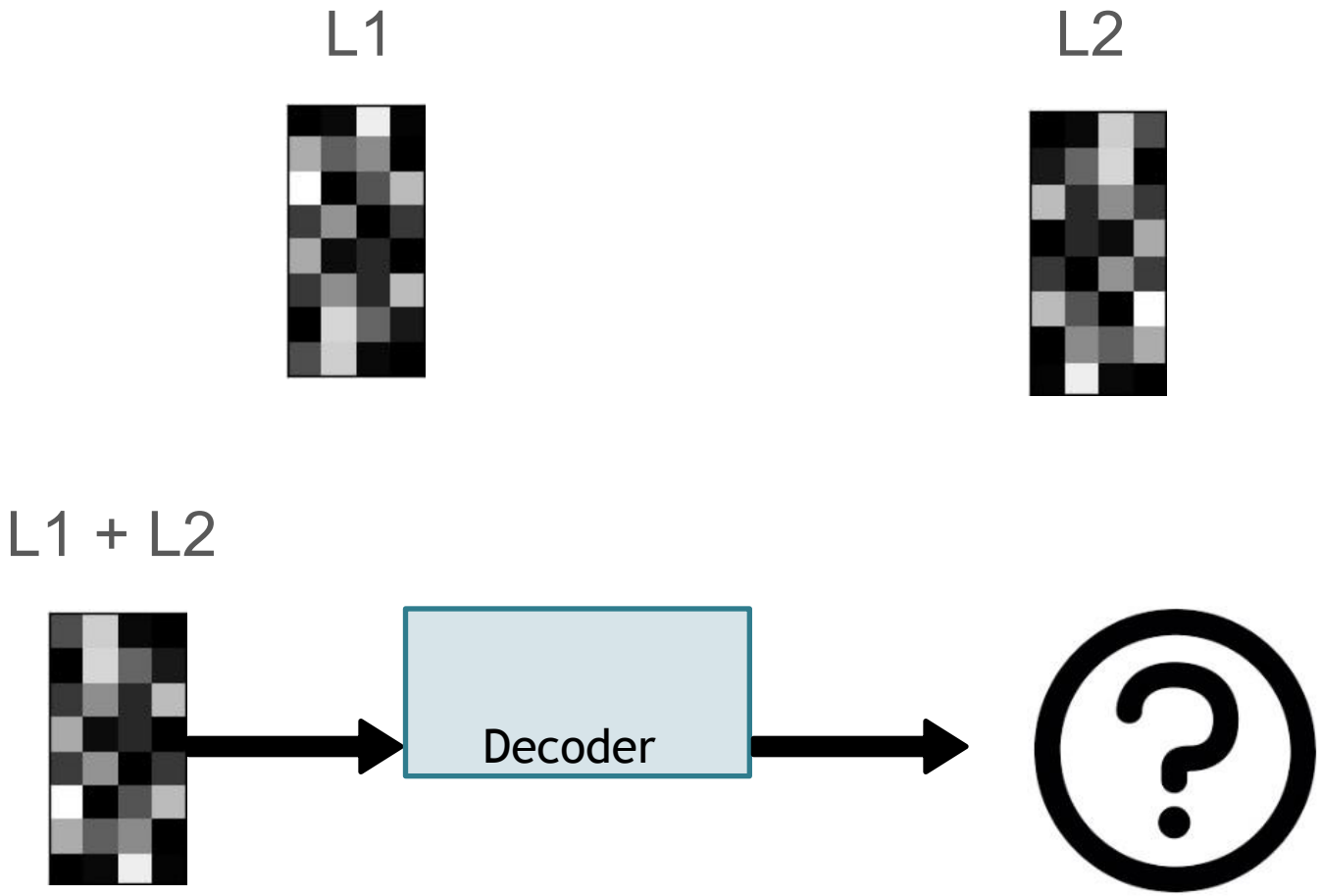


Figure 8.10 A continuous space of faces generated by Tom White using VAEs

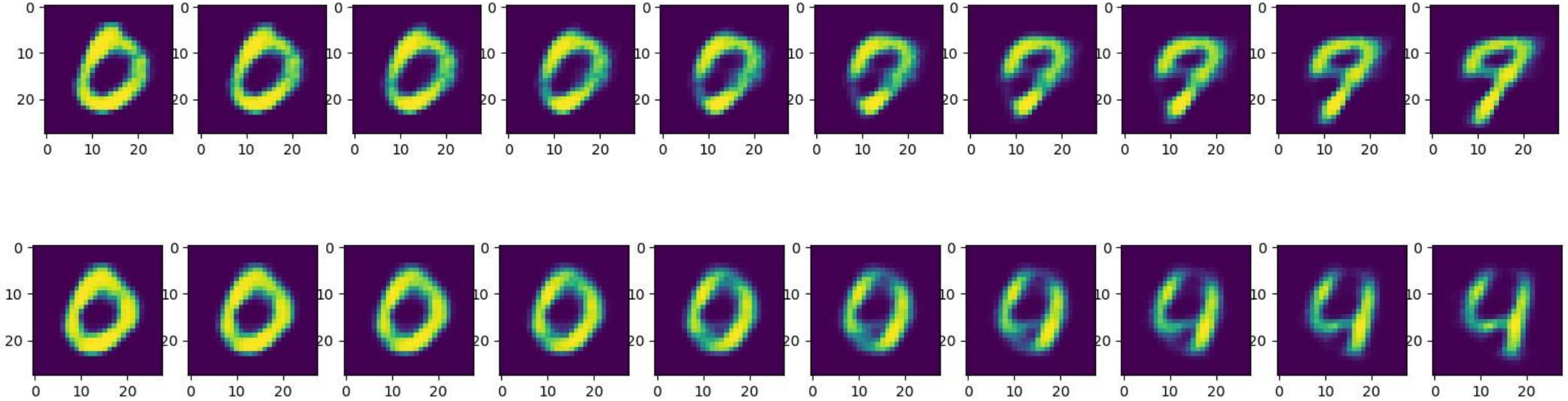
Simples interpolação no espaço latente - keras



Simples interpolação no espaço latente - keras



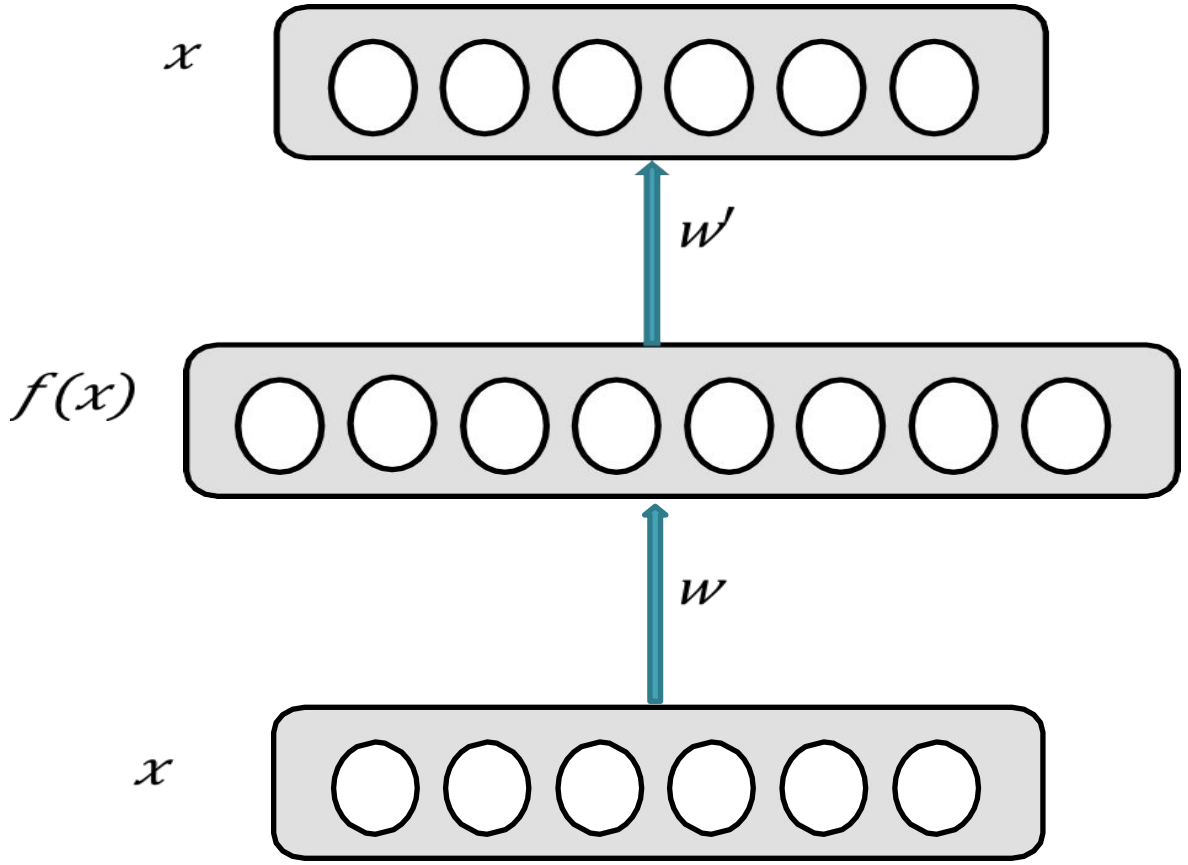
Simples interpolação no espaço latente - keras



https://blog.keras.io/building-autoencoders-in-keras.html

Autoencoder sobrecompleto

- Camada escondida é **Sobrecompleta** se maior que a de entrada
  - ❑ Sem compressão na camada intermediária, mas usualmente acompanhada de regularização agressiva.
  - ❑ Cada neurônio intermediário pode isolar e especificar melhor o conjunto de informações originais.
- Adicionar dimensões é útil para treinar um classificador linear e computar métricas de similaridade.
- Um código em dimensão maior ajuda a modelar distribuições mais complexas
- Entretanto… não há garantias que unidades intermediárias consigam extrair bons atributos.



Autoencoder com filtragem de ruído

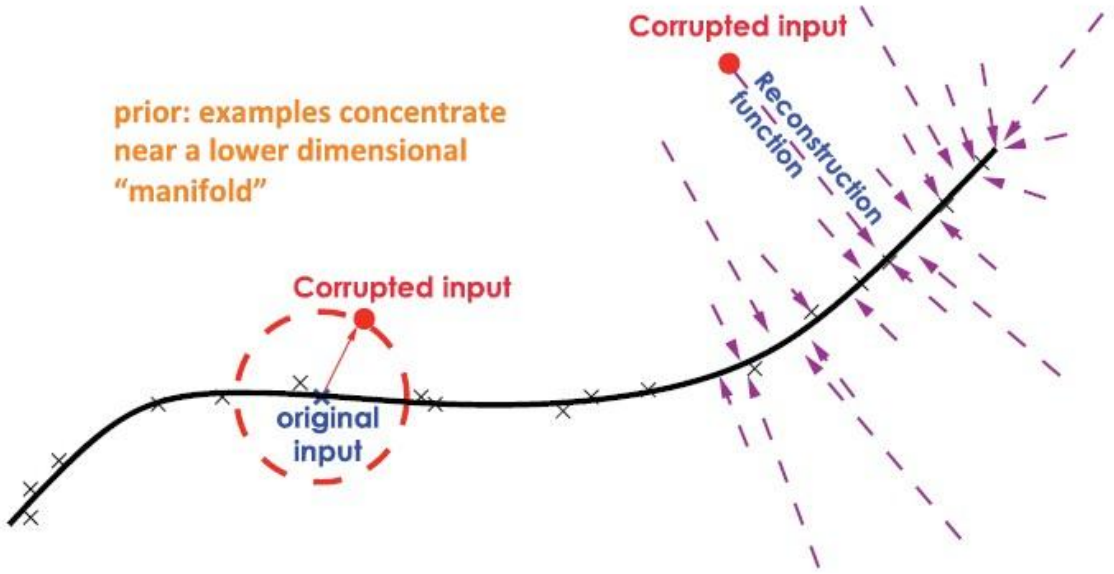
(*denoising autoencoder*)

- Ideia básica: representação aprendida seja robusta a ruídos nos dados de entrada.
- Aplicar processo probabilístico em cada exemplo de treinamento **x** antes de apresentá-lo à rede.
- Alternativas
  a) com probabilidade p, atribuir zero a cada componente de **x**.
  b) perturbar cada componente de **x** por meio de um ruído gaussiano aditivo.

Representation Learning - A Review and New Perspectives (2013)

Autoencoder com filtragem de ruído

(*denoising autoencoder*)



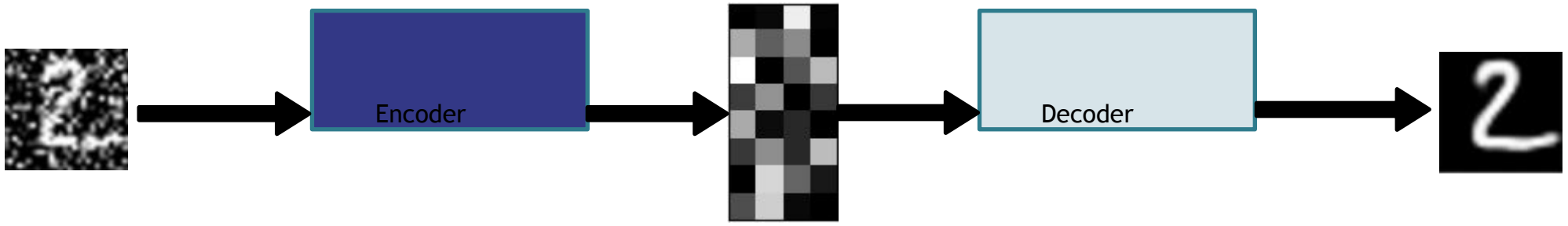


Representation Learning - A Review and New Perspectives (2013)

Denoising Autoencoders

- Codificar a entrada sem "copiar" a função identidade
- Desfazer o efeito do processo estocástico de corrupção aplicado na entrada.

Modelo mais robusto a ruídos



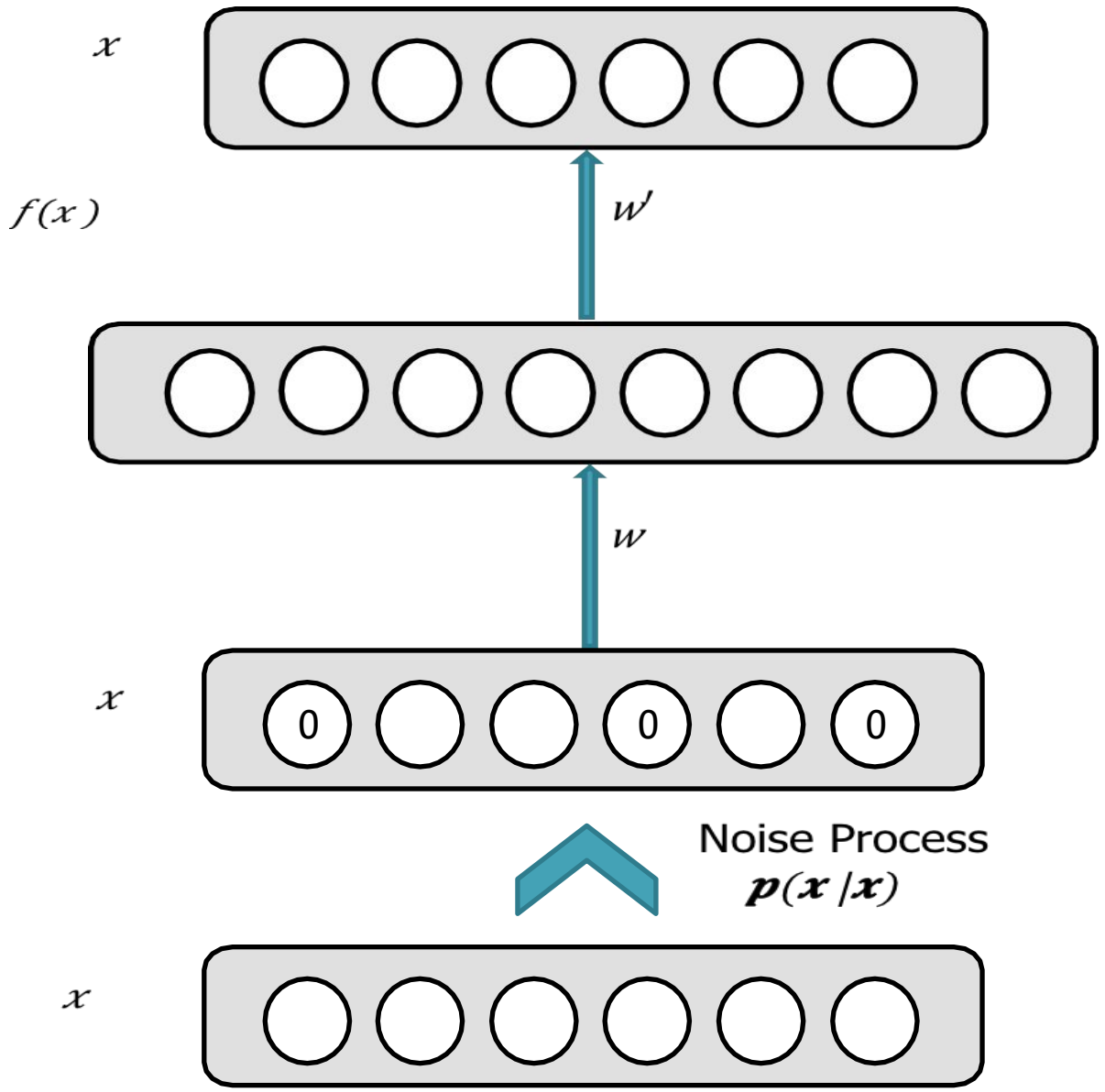
Entrada ruidosa

Representação no espaço latente

Entrada de-ruidosa

Denoising Autoencoders

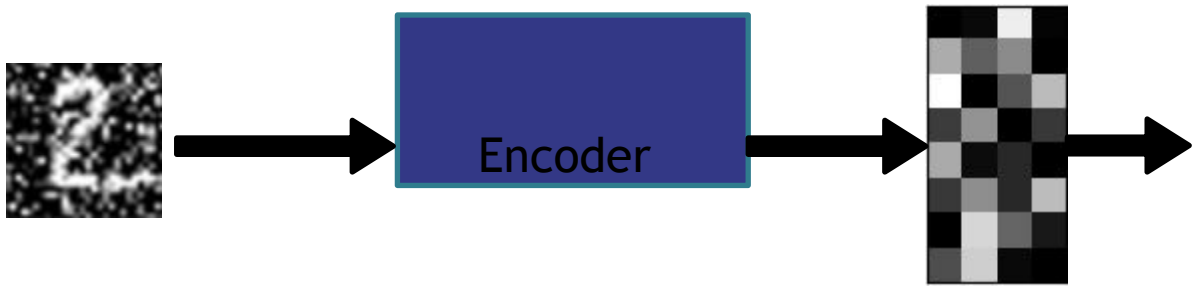
- Autoencoder recebe uma entrada com ruído sintético adicionado a x.
- No treinamento, o autoencoder tenta reconstruir x sem ruído.
- Estamos forçando o autoencoder a aprender como remover o ruído da entrada dos dados.



Denoising Autoencoders (DEA)

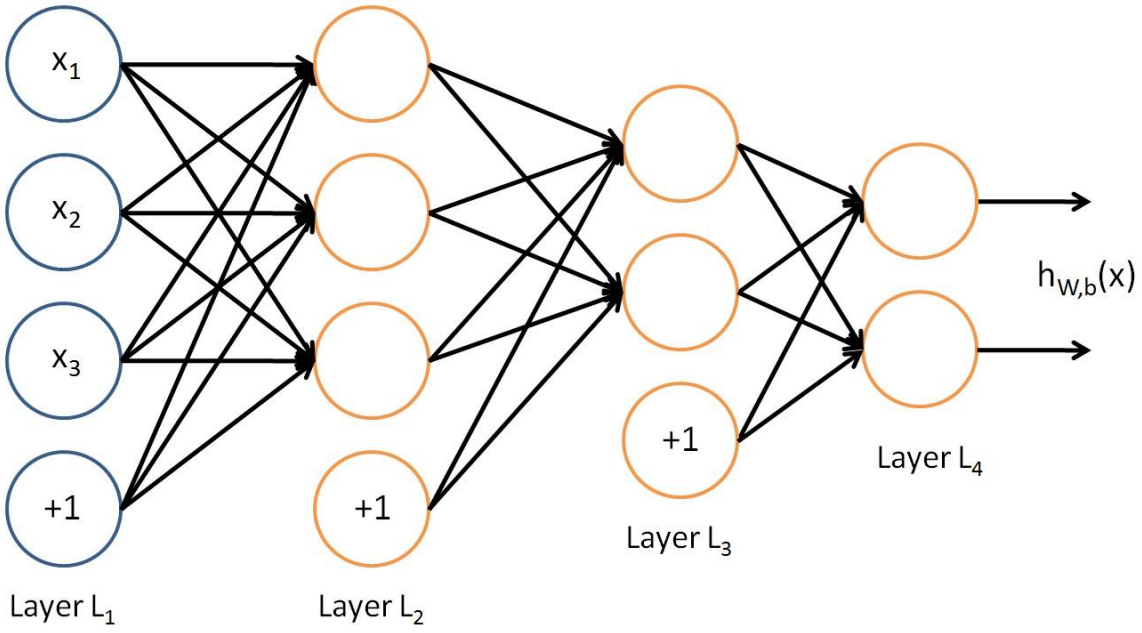
Caso de uso:

- Além de gerar uma versão sem ruído...
- Extrair representações robustas para um classificador



Entrada ruidosa

Representação no espaço latente



Processo - Denoising Autoencoders (DEA)

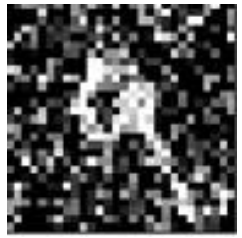
Dada uma entrada



Aplique ruído



x



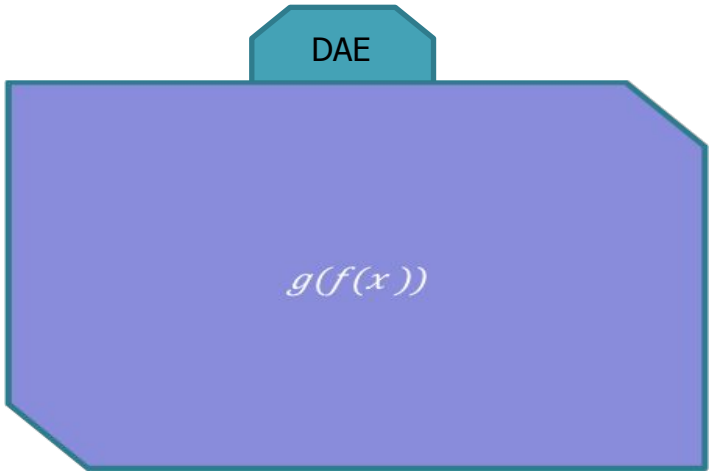
Processo Denoising Autoencoders (DAE)

x

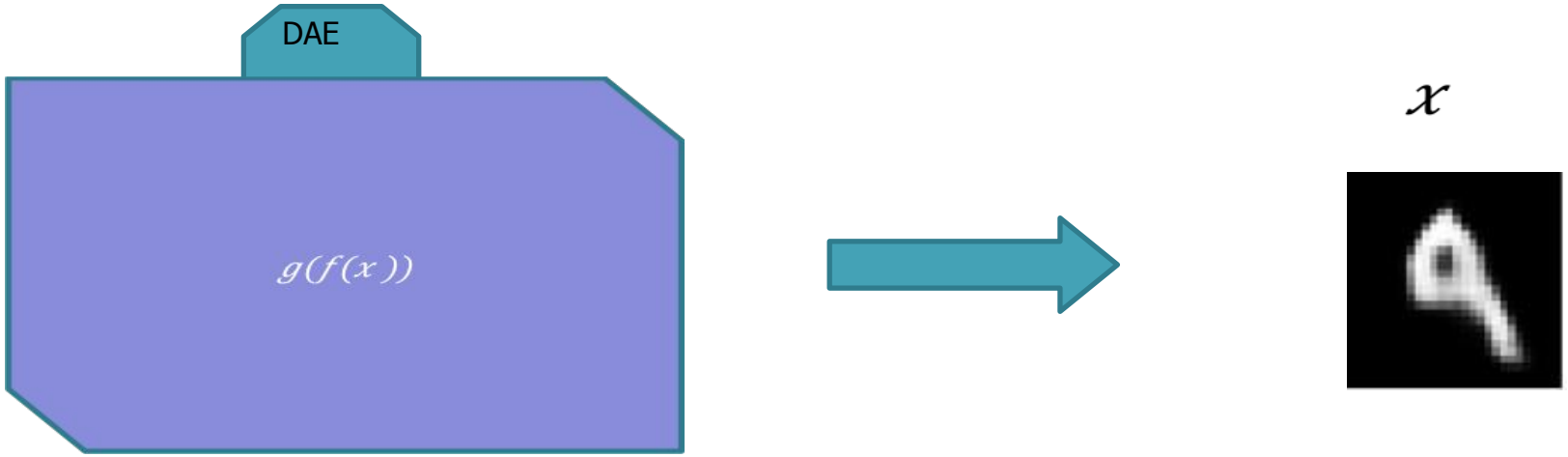


Aplique no autoencoder DAE



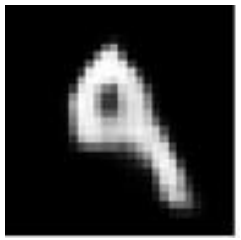


Processo - Denoising Autoencoders



Processo - Denoising Autoencoders

x



Compare (com a função de perda)



x



Autoencoder esparso

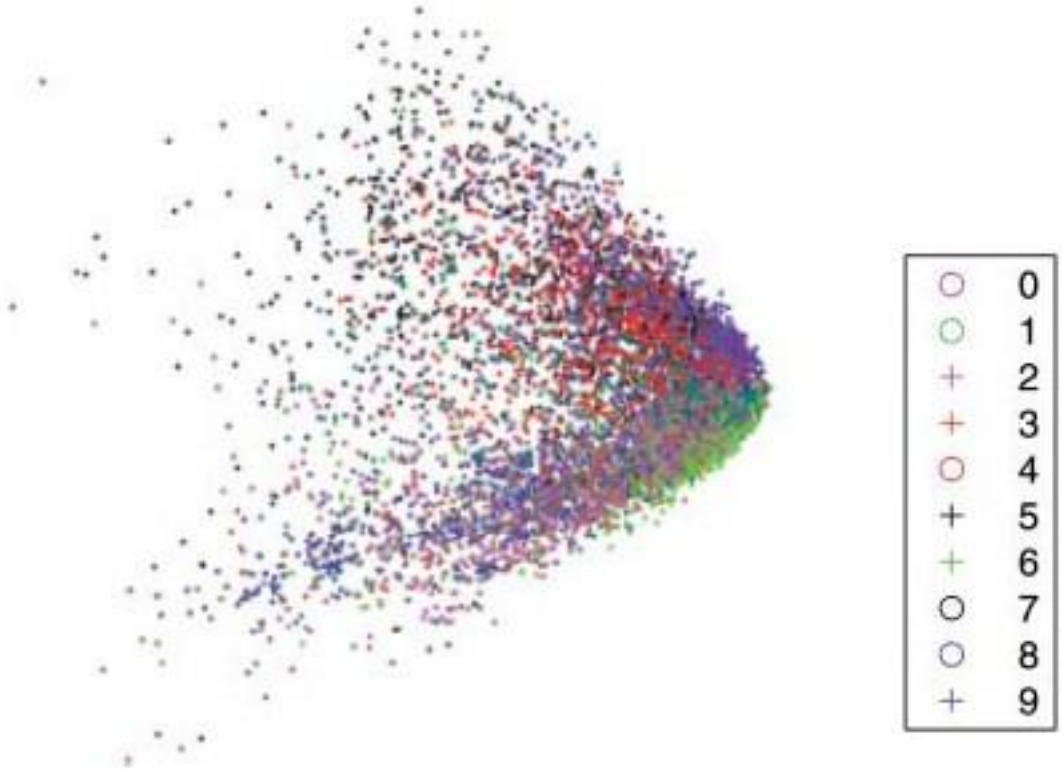
(sparse autoenconder)

- Objetivo: apenas pequena quantidade de unidades da camada oculta seja ativada para cada padrão de entrada.

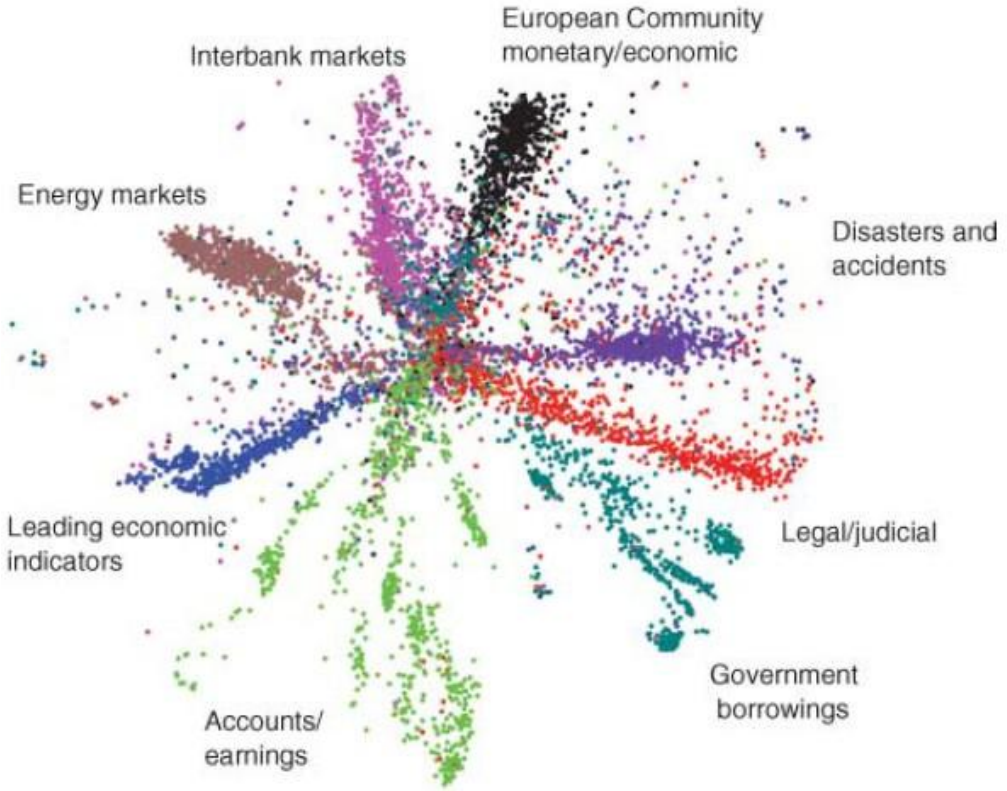
- Esparsidade pode ser obtida
  - Termos adicionais na função de perda durante o treinamento (regularização l1)
  - Forçar apenas as k unidades mais ativas e tornando todas as demais unidades iguais a zero

Aplicações - Visualização/Redução de Dimensionalidade

PCA
(k=2)

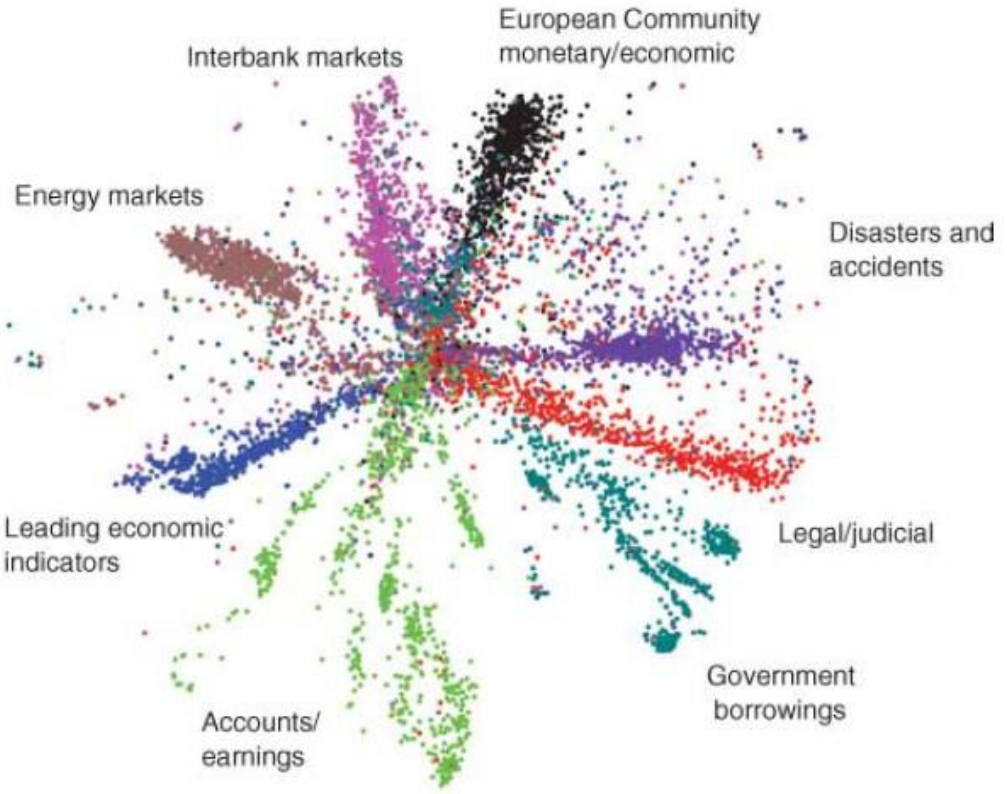


Autoencoder
(2000-500-250-125-2)

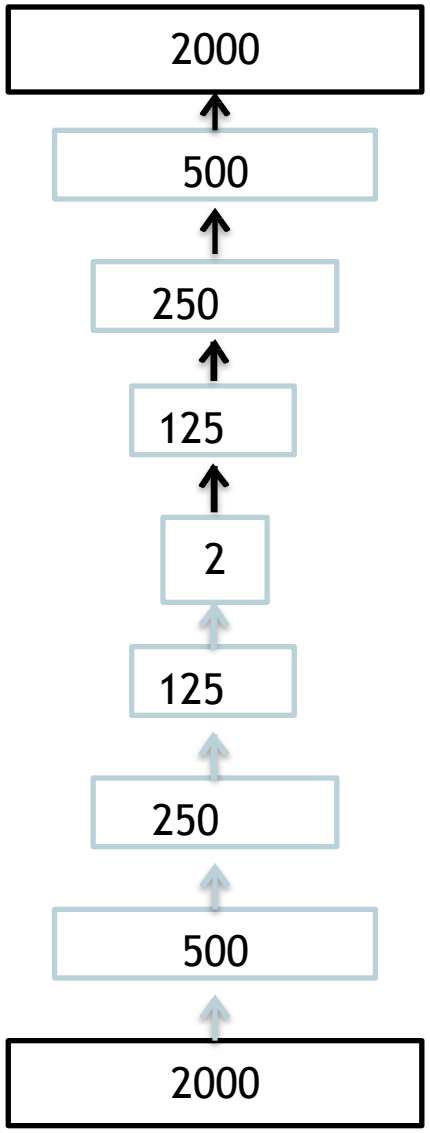


Reducing the Dimensionality of Data with Neural Networks (Science, 2006)

Aplicações - Visualização/Redução de Dimensionalidade



Reducing the Dimensionality of Data with Neural Networks (Science, 2006)



Aplicações – aprendizado/visualização de conceitos

- Uma Imagem extraída para cada vídeo do YouTube (10 milhões de imagens)
  - Imagens (200x200)
- A rede aprendeu conceitos de face humana e de face de gatos

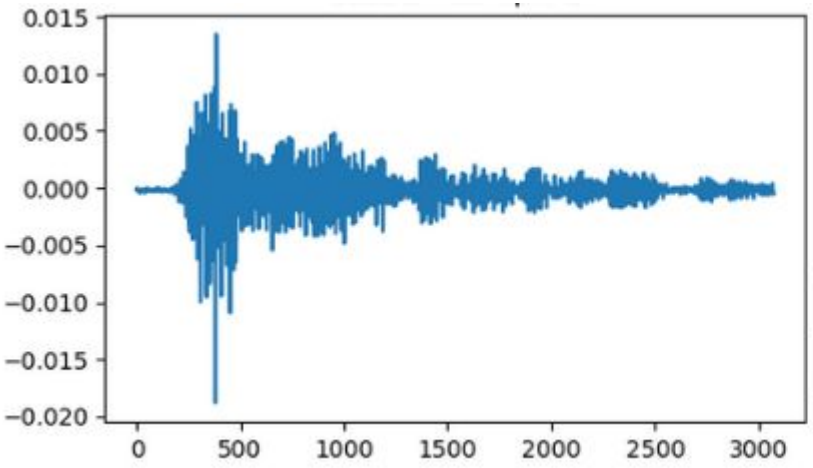


Le et al, Building high-level features using large-scale unsupervised learning, 2012

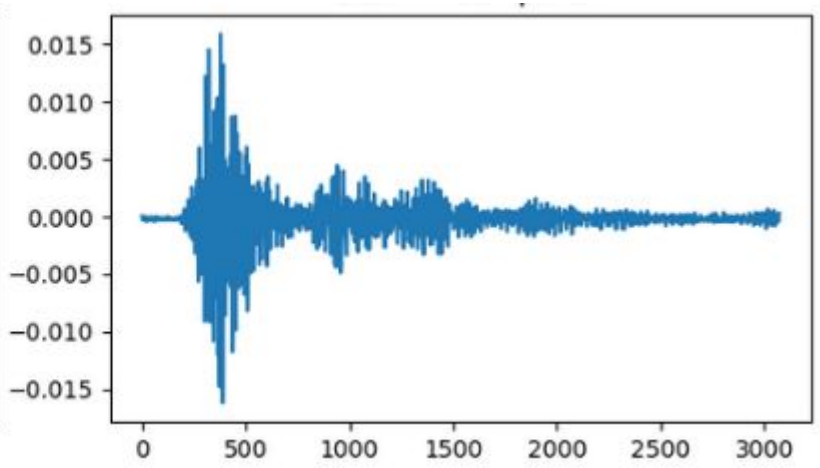
Aplicações – Detecção de Anomalias

- Dado um sinal de sensores acústicos em tubulações de óleo, há alguma anomalia/defeito na tubulação?
  - Problema: quase todas as tubulações estão em bom estado.

Anomalia

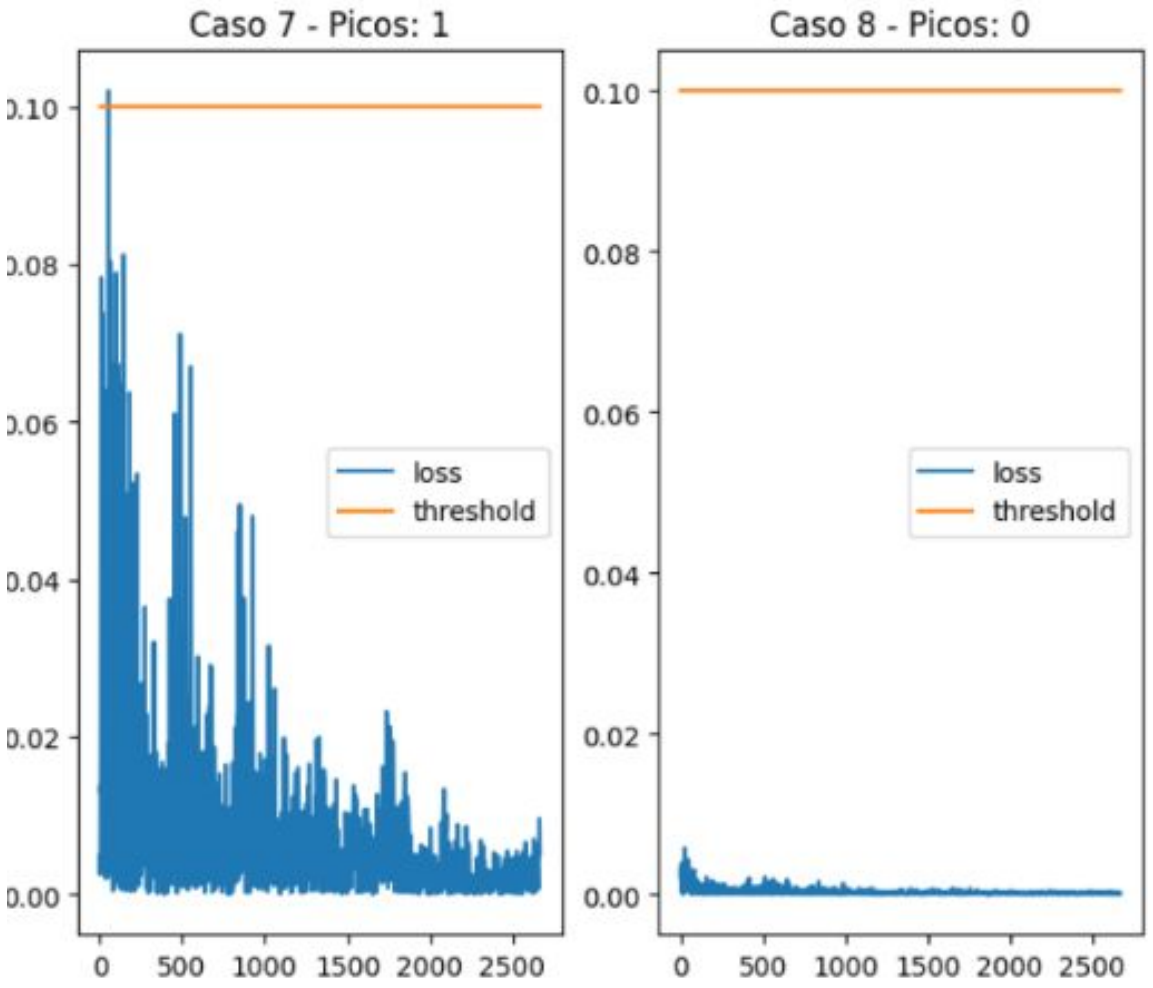


Normal



Aplicações – Detecção de Anomalias

- Aplicando um autoencoder LSTM treinado apenas em tubulações normais:
  - Erro de reconstrução do LSTM é grande em tubulações anômalas.



Autoencoders: Idéia simples, antiga e robusta!

- Com restrições colocadas na rede (limitar # de neurônios ou usar regularização) a rede pode descobrir a estrutura dos dados.
  - Captura relações entre as variáveis que explicam a distribuição dos dados.
  - E tudo, sem utilizar rotulação!

Referências

1. François Chollet. Deep Learning with Python. Manning publications. 2019
2. https://arxiv.org/pdf/1206.5538.pdf
3. https://blog.keras.io/building-autoencoders-in-keras.html
4. https://towardsdatascience.com/step-by-step-understanding-lstm-autoencoder-layers-ffab055b6352
5. http://www.deeplearningbook.org/contents/ autoencoders.html
6. http://deeplearning.net/tutorial/dA.html
7. http://ufldl.stanford.edu/tutorial/unsupervised/ Autoencoders/
8. http://ufldl.stanford.edu/wiki/index.php/ Stacked Autoencoders
9. http://www.jmlr.org/papers/volume11/vincent10a/ vincent10a.pdf
10. https://codeburst.io/deep-learning-types-and- autoencoders-a40ee6754663